

MediSync SA

Digital Healthcare Management System

FINAL CONSOLIDATED REPORT

NPRT63 — Project (Phase 4)

Group Members:

Simamkele Jokose — 202424701 (Group Leader)

Nokukhanya Mkhize — 202407968

Nontethelelo Gumede — 202410803

Rebecca Tshelane — 202412895

Lecture: Silas Verkijika
Due Date: Monday, 12 October 2026
Weight: 50%

Contents

INTRODUCTION 3
BACKGROUND 4
THE PROBLEM IN SOUTH AFRICAN HEALTHCARE 4
PROBLEM STATEMENT: 5
THE KEY FIVE CHALLENGES 5
PHASE 1: 6
PROJECT PROPOSAL SUMMARY 6
PROPOSED SOLUTION OVERVIEW 6
PROJECT OBJECTIVES 6
MVC ARCHITECTURAL OVERVIEW 7
PHASE 2: 7
REQUIREMENTS SPECIFICATION SUMMARY 7
DESIGN SPECIFICATION SUMMARY 8
PHASE 3: 9
DESIGN SPECIFICATION (SUMMARY) 9
PHASE 4: 11
TECHNOLOGY STACK(Implementation) 11
SYSTEM ARCHITECTURE 12
CODE QUALITY 24
TESTING 25
REFLECTION ON ACHIEVEMENTS 27
COMPREHENSIVE DOCUMENTATION 30
CONCLUSION 30
REFERENCES 32

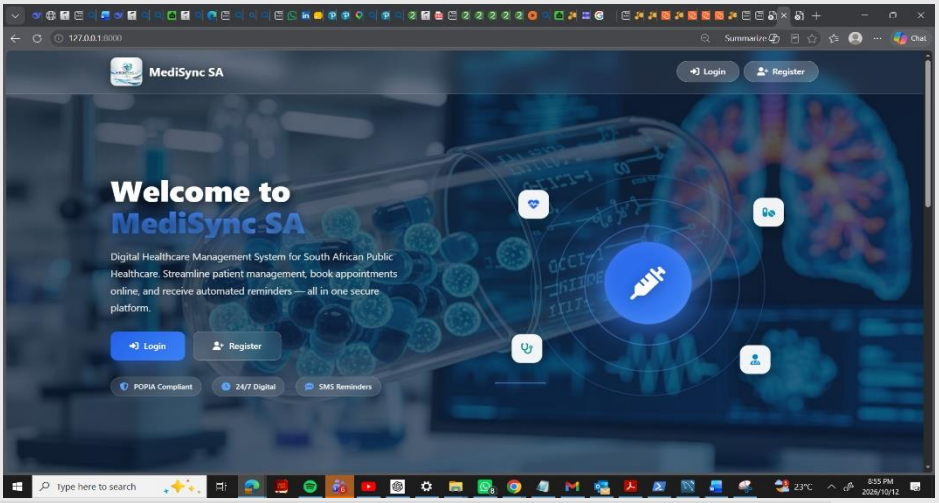
INTRODUCTION

MediSync SA is a web-based healthcare management system developed to replace the manual patient records and appointment booking process currently used in South African public healthcare facilities with a secured digital alternative. It was a year-long project completed in four phases. Phase 1 focused on identifying the problem and solution, while Phase 2 centred around requirements specification and UML design. The system's design was then refined in collaboration with users and the database documented in Phase 3. Finally, the current existing Phase 4 (which is the last phase), which focuses on completing the implementation and documenting the process and results, culminates the entire project and reflects the achieved outcomes.

This report compiles the information from all phases into a singular concluding document. Firstly, it serves to note that the system designed in Phase 2 and specified in Phase 3 was successfully implemented and is fully functional. Secondly, it provides extensive documentation on the implemented system. The report contains documentation of the working system in the form of photos of the code, database structure, and tests conducted. However, its primary content concerns the description of the new features and their impact. It is in the last phase that the project attains its objective of demonstrating the successful realisation of a working system. As such, Phase 4 is the most weighted of all phases at 50% and did not involve any additional requirements – only the completion of the previously designed system and presentation to the supervisor.

The web application was built using Django (Python 3.13), MySQL 8.0, Bootstrap 5, and ClickSend. The latter is used in two places: first, to send SMS with OTP to phone numbers provided by the user upon logging in the system, and, secondly, sending out reminders 72 hours, 24 hours, and 2 hours before confirmed appointments. The system offers two portals: Staff/Administrator, which allows the management of the records of all users, patient queues, and appointments, and Patient, which allows the patient to enter their details and book an appointment. There are five types of users within the system: Patient, Doctor, Nurse, Medical Receptionist, and Administrator, each with their own dashboards.

In addition to features requested by the users, two additional features were added, which could be implemented through the Django admin panel: public holiday blocking and doctor leave blocking. They serve to prevent the system from suggesting slots during which the doctors are not present at the facility. They can be scheduled through Django admin panel and appear in red colour on the calendar. The functionality is beneficial because it allows the clinic to adapt to circumstances without requiring technical support. For instance, if a public holiday falls on a weekday, or a training day for the doctors is scheduled, or a leave is planned, the administrators can block the dates and inform the patients through the SMS bot.



1.Home page-

Of MediSync SA

BACKGROUND:

THE PROBLEM IN SOUTH AFRICAN HEALTHCARE

South African public health facilities have been using paper-based systems for managing patient records and making appointments for years. Clinics have been filing patients' medical records and manually marking appointments. This practice has been in place for a long time, and the consequences of this approach have been widely reported.

Paper medical records can be lost or misplaced leading to dangerous situations where doctors may treat a patient based on incorrect information. This compromises not only patient safety but also the security of the medical records. Patients with chronic illnesses such as HIV, tuberculosis, diabetes, hypertension, and epilepsy need to regularly visit clinics for treatment and must not miss any scheduled appointments because of the potentially debilitating effects of the untreated conditions. Furthermore, the inability to quickly access patient medical records leads to poor record-keeping, where doctors are forced to create new records when old ones cannot be found, and patients receive treatment based on spurious medical information. Searching for paper medical records is an extremely cumbersome process involving rummaging through filing cabinets and takes, on average, over 3 hours. Another implication of the reliance on paper-based systems is the inability to book appointments remotely. Since many South Africans do not have access to reliable transport and are forced to report to clinics in person, many rural, elderly, disabled, and working patients miss appointments due to the lack of reminders. Lastly, the inability to digitally input appointments means that no reminders can be sent to patients, resulting in excessive no-shows and wasted medical staff time.

The South African government is fully aware of the challenges that the paper-based system is facing and is currently in the process of digitizing patient medical records. The Gauteng Department of Health (GDoH) has made remarkable progress with its initiative to digitize the paper patient records in public health facilities in the province. They have kicked off their pilot project in 2024 and digitized about 800,000 patient files at Chris Hani Baragwanath Academic Hospital, the largest hospital in Africa, and Rahima Moosa Mother and Child Hospital, which is a leading women's hospital in South Africa. Using the cloud-based eHealth technology, GDoH aims to create an integrated digital health information system where patients' records will be stored safely and securely. By mid-2025, GDoH aims to digitize 800 million patient records in 37 public hospitals in the Gauteng province over 36 months in key specializations such as maternity, trauma, mental health, and chronic illness. At the national level, the National Department of Health has launched the Health Systems Digital Information program as part of the overall Health Insurance Programme and is currently developing an interoperable Health Patient Record System that will give patients a portable electronic health record. The Council for Scientific and Industrial Research is also buiding digital products for the National Health Insurance, including the Health Patient Registration System, which will establish a common standard patient registration in all public health institutions.

Regardless of the efforts of the South African government to digitalize the information storage of patients' medical records and book appointments, such a system is not yet fully applied nationwide, and many public health facilities, especially clinics in rural areas, remain analog. A study published in 2021 that performed a systematic review of electronic health records and their implementation in South Africa's public health facilities identified technical, social, and environmental barriers to the adoption of EHR, including the lack of supporting infrastructure, insufficient training and commitment from users, political challenges, and the absence of a framework for implementation and management. Another study published in 2020 that analyzed the complex factors associated with the

stalled implementation of a ground-up EHR system at a South African hospital concluded that the outer setting factors, including governance, funding, and policy, could negatively affect an EHR implementation if not carefully considered. These studies emphasize the importance of bridging the gap between the national level initiative to digitize public health information and the practical needs of the local level. This is the focus area that MediSync SA tackles all the healthcare issues by providing a self-contained website application that does not require much in terms of additional infrastructure. It aligns with the needs of individual clinics and hospitals, which are different, and delivers a scalable solution that could be estiblished at various levels of the health care system.

Moreover, it was developed in a way that the use of SMS reminders is an effective method of reminding patients of forthcoming appointments. An organized review and meta-analysis evaluating the success of short message service (SMS) reminders for pediatric appointments concluded that SMS reminders were a cost-effective procedure of enhancing appointment attendance. Additionally, studies examining the effectiveness of SMS reminders in appointment-keeping behavior demostrated that the use of SMS reminders improved appointment attendance by 14-25% in low-resource settings. MediSync SA has intergrated an SMS reminder system as a component of its appointment management system to lower the number of patients arriving late or sometimes not at all to scheduled appointments. Patients get their SMS cell phone reminders 72 hours, 24 hours, and 2 hours before their scheduled appointments.

PROBLEM STATEMENT:

THE KEY FIVE CHALLENGES

MediSync SA was developed to address five key issues affecting healthcare facilities, patients, and medical staff. The first problem is the **loss of patient records** which affects patients, nurses, doctors, and administrators, increasing health risks since patients' records are not retrieved in good time, especially for patients with chronic conditions. The second problem is the **duplication of patients' records**, making it difficult for doctors and nurses to access patients' medical history conveniently. The third challenge is the **long waiting time** for patients which affects the workflow of the whole healthcare facility, whereby patients can wait for more than three hours to see a doctor. Also, the **in-person visit booking** affects rural patients, elderly patients, disabled patients, and busy patients and causes them to incur extra costs and time, waste patients' time, cause patient absenteeism at work, and lower the chances of getting care, especially if the patients are too old or too young. Finally, **missed appointments** have a significant effect on doctors and managers, whereby they waste a considerable amount of time when a patient does not show up after booking the appointment.

MediSync SA will address these issues by addressing patients' records, appointment system, and waiting time for patients.

#	Problem	Affected Users	Impact
1	Lost or misplaced patient files	Patients, nurses, doctors, admin	Delayed consultations, incomplete medical history
2	Duplicate records	Admin staff, doctors	Fragmented medical history
3	Long waiting times	All patients	Patients wait more than 3 hours

4	In-person booking only	Rural, elderly, disabled, working	Extra cost and time for patients
5	High missed appointment rates	Doctors, facility managers	Wasted clinical time and resources

PHASE 1:

PROJECT PROPOSAL SUMMARY

Phase 1 presented the background of the project. It focused on the South African public healthcare context integrated with the issues brought by the paper-based system. From this part of the project, the project objectives emerge and lead various aspects of the project. There were 7 major objectives that the project will achieve which includes a centralized database for patient records, instant search of patient records including South Africa ID number, doing away with paper folders, online booking of appointments with green and red indicators for availability and patients can cancel or reschedule appointments, automatic SMS reminders and make the whole system POPIA compliant. The major problems that the project will address were also identified and each one was discussed as found in the previous section.

PROPOSED SOLUTION OVERVIEW

The proposed solution is a web-based healthcare management system with two portals; a Staff and Administrator portal and a Patient portal. The Staff and Administrator use the Staff portal to add or update patient records, add or update appointments including the management of the waiting list. The Patient portal is used to allow patients to register to the system and make appointments online and receive automatic SMS reminders. In addition to the two portals, the proposed system will also include a digital queuing system, role-based access control which will support 5 types of users, avoid duplicate records by allowing the search of patient records using South Africa ID, a colour-coded online appointment booking facility with green and red indicators to show availability and bookings respectively, and the system will ensure POPIA compliance to protect patient information.

Some goals were identified against which the proposed solution will be measured. A web-based solution will provide effectiveness since it allows instant search and retrieval of patient records using South Africa ID. The system will provide efficiency by allowing fast registration, updating of patient records and management of appointment bookings. It will also increase safety by avoiding the use of paper folders hence reducing exposure and loss of information, avoiding duplication of records by searching using South Africa ID and limiting access to authorized personnel. Fast access to the system and features such as instant search and retrieval of patient records and appointments will reduce waiting time at the hospitals.

PROJECT OBJECTIVES

The objectives of the MediSync SA project are as follows:

- To achieve a central digital database for storage of patient records to overcome the use of paper files.

- To develop a solution to a rapid search and retrieval of patient records using South Africa ID numbers.
- To develop a solution to the use of paper folders in storing of patient records.
- To facilitate the patient appointment booking online
- A colour-coded online appointment booking facility which includes an automatic SMS reminder to booked patients.
- Ensure compliance with the Protection of Personal Information Act (POPIA).
- Improve the efficiency of healthcare delivery through the elimination of the use of paper folders and provision of a rapid record retrieval and appointment booking facility.

MVC ARCHITECTURAL OVERVIEW

The Model-View-Controller (MVC) framework was chosen. In this framework, the Model stores information about the system's state, encapsulates the business rules, checks for duplicates, booking specifications, and access control. The View displays patient's records, appointments, and calendars and sends messages to the controller. The Controller receives and handles the staff and patient requests, including processing the login request and records management, appointment management, and queue management, interprets the requests, updates the models, and directs the users to the appropriate page.

PHASE 2:

REQUIREMENTS SPECIFICATION SUMMARY

Phase 2 formalised the functional and non-functional requirements and produced the complete set of UML behavioural models of the system that would guide the implementation phase. The key deliverable in Phase 2 is critical as it offers the implementation team design specifications derived from abstract project objectives described in Phase 1.

Functional requirements encapsulate the system features that determine the types of services it will offer. For instance, regarding patient registration and authentication, the system should allow patient registration with their 13-digit South African ID number, verify the authenticity of the SA ID using the Luhn checksum algorithm, avoid dual registration of patients with the same SA ID, allow staff members to search for patients using their SA ID, and authenticate staff members with their SA ID with OTP. With regard to appointment management, patients should be able to book appointments online, filter available and unavailable time slots with a colour-coded calendar, avoid double-booking of time slots, allow staff members to create, update, and cancel appointments, and send SMS reminders to patients 72 hours, 24 hours, and 2 hours before appointments. In terms of clinical functionalities, doctors should be able to create digital prescriptions associated with a patient's record, allow staff members to create, update, and delete patient records, link patient medical history with a patient ID, ensure retrieval of a patient's record within 5 seconds of searching, allow staff members to manage a digital waiting queue, and record all user activities in an audit log. Finally, for security and access control, the system should deny users access to functionalities not permitted to their clearance level and record all users' activities in an audit log.

Non-functional requirements describe system-level constraints or attributes that offer guidelines on how to build or operate the system. In terms of performance, the system's search feature should return results within 5 seconds of query initiation, support over 100 concurrent users on its platform, and

remain performant. In terms of security, the system should comply with the Personal Information Protection Act (POPIA), protect patient information with secure OTP verification, and protect all patient information with robust encryption algorithms. With regard to usability, the system should be intuitive and easily navigable by first-time users with minimal training. As far as reliability is concerned, the system should operate 24/7 with a 99% uptime and recover lost data within 5 minutes after a system failure. In terms of scalability, the system should be scalable to serve multiple hospitals, without redesigning the system, and able to scale up as more patient records are added to the database. When it comes to maintainability, the system should be developed using a modular approach to ease system maintenance, updates, and debugging and designed using the Model-ViewController (MVC) architectural framework. Finally, in terms of compatibility, the system should be operable 24/7 through web browsers, be accessible via mobile devices, be compatible with all standard web browsers, and be integrated with a third-party SMS service gateway

The UML behavioural models capture the complete system behaviour from the perspective of different stakeholders. Firstly, the Use Case Diagram shows five actors (Patient, Doctor, Nurse, Medical Receptionist, and System Administrator) and the permitted functionalities that each actor is allowed to perform in the system. The Activity Diagram depicts the procedural steps that the system will perform to enable the process of booking appointments. Hence, the diagram shows how the system will make available time slots for selection and what actions to take if the chosen time is not available. The State Diagram illustrates the appointment process states (SlotCreated->Available Green->Pending->BookedRed->InProgress) and actions that trigger state transitions. The Class Diagram shows all the classes (User, Patient, Staff, MedicalReceptionist, Nurse, Doctor, SystemAdministrator, Appointment, MedicalRecord, Prescription, DigitalQueue, AuditLog), attributes, methods, and relationships between classes. The Sequence Diagram demonstrates a doctor searching for a patient's medical record by SA ID. The Component Diagram shows the presentation layer, application layer, database layer, and external components. The Deployment Diagram depicts how user devices connect to the web server through HTTPS, how the web server connects to the application server, how the application server connects to a database server, and the presence of an external SMS component.

The Work Breakdown Structure below contains seven deliverables: Project Management, Analysis and Design, Backend Development, Frontend Development, Integration, Testing and QA, and Deployment and Training. The Gantt chart shows a 15-week project plan. The project will begin in March 2026 and be completed in June 2026. The backend development will be completed by 30th April, whereas the frontend development will be completed by 5th June.

UML DIAGRAMS AND THE PURPOSE FOR EACH

- **Use Case Diagram:** Permission and responsibility of 5 actors.
- **Activity Diagram:** Appointment booking process including decision points.
- **State Diagram:** The appointment booking lifecycle with states.
- **Class Diagram:** The system's structure including 12 entities and their attributes and methods.
- **Sequence Diagram:** Doctor's record search and retrieval of a patient's record.
- **Component Diagram:** Layers including Presentation, Application, Database, and External.

Deployment User's devices → HTTPS → Web/App/DB servers + SMS Gateway

DESIGN SPECIFICATION SUMMARY

WORK BREAKDOWN STRUCTURE AND GANTT CHART

The Work Breakdown Structure was divided into seven categories: Project Management, Analysis and Design, Backend Development, Frontend Development, Integration, Testing and QA, and Deployment and Training. The 15-week Gantt Chart illustrated the timeline of the project commencing in March 2026 and ending in June 2026. it also followed the project's key milestones, which included the completion of the backend and frontend developments and overall system deployment.

PHASE 3:

DESIGN SPECIFICATION (SUMMARY)

USER FEEDBACK AND DESIGN ITERATION

Phase 3 saw the refinement of the system design based on the feedback provided by the users during the testing phase. The feedback was provided through questionnaires, interviews, and User Acceptance Testing sessions conducted with representatives of all the user groups: patients, receptionists, doctors, nurses, and administrators. The purpose of feedback collection was to obtain insights into the extent to which the system was intuitive, user-friendly, and efficient for a healthcare setting.

The patients identified the "Book appointment" option on the dashboard as hard to locate and access due to its small size and the distance between the "Back to previous page" button and the "Book appointment" banner. They requested changes to ensure that the SMS reminders included the address of the clinic and the time of the scheduled appointment in a specific manner. The receptionists expressed dissatisfaction with the current search options, requesting additional search fields, including the SA ID and surname. Doctors and nurses indicated that the text fields for prescriptions and diagnosis were too narrow, proposing that they be self-saved to avoid losing crucial patient information when switching tabs or minimizing the application. Finally, the administrators recommended that the dashboard be redesigned because of the excessive menus that make the system overwhelming for first-time users.

Based on this feedback, a series of redesigns were undertaken, including making the "Book appointment" button bigger and moving it to the top-right corner of the dashboard. It was also changed to green to improve its visibility. The search function was enhanced by adding the SA ID, surname, and appointment number as searchable items. The text fields for prescriptions and diagnosis were made wider and self-saved. The menus were organized into groups based on their functions, such as Patient Management, Appointments, and Reports. Finally, the SMS reminder was modified to include the address of the clinic and the name of the doctor, in addition to the existing information.

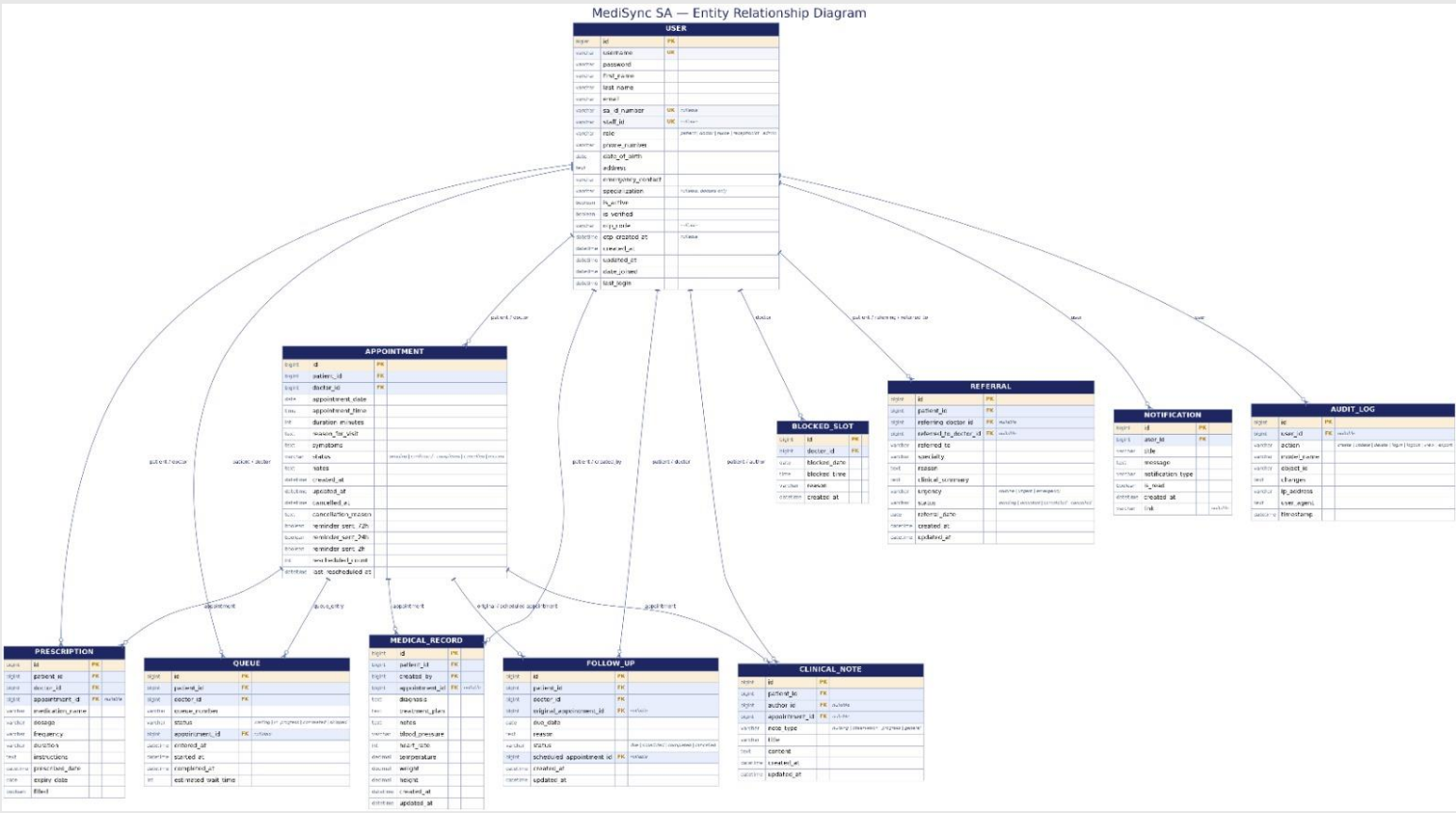
Entity Relationship Diagram

-Phase 3 documented the complete Entity Relationship Diagram. The diagram reflects the final database structure including **core_user, core_appointment, blocked_slots, core_queue, core_medicalrecord, core_clinicalnote, core_prescription, core_referral, core_followup, core_notification, and core_auditlog.**

The MediSync SA — Entity Relationship Diagram showing User at the top centre and Appointment, Referral, Blocked Slot, Notification, Audit Log, Prescription, Queue, Medical Record, Follow-up, and Clinical Note branching from it

PLEASE NOTE ER DIAGRAM HAS BEEN UPDATED TO ADD MORE FEATURES

ER Diagram



The following table explains how each entity in the database connects to another entity, along with the type of relationship and the justification for its existence. Every relationship listed below is drawn directly from the Entity Relationship Diagram, using the exact table names shown in the diagram.

Relationship	Type	Justification
user → appointment (as patient)	1:*	One user (as patient) can book many appointments over time
user → appointment (as doctor)	1:*	One user (as doctor) can conduct many appointments
user → prescription (as patient)	1:*	One user (as patient) can receive many prescriptions
user → prescription (as doctor)	1:*	One user (as doctor) can issue many prescriptions
user → queue (as patient)	1:*	One user (as patient) can appear in the queue many times
user → queue (as doctor)	1:*	One user (as doctor) can be linked to many queue entries

user → medical_record (as patient)	1:*	One user (as patient) can have many medical records
user → medical_record (as created_by)	1:*	One user (as doctor or nurse) can create many medical records
user → follow_up (as patient)	1:*	One user (as patient) can have many follow-up records
user → follow_up (as doctor)	1:*	One user (as doctor) can create many follow-up records
user → clinical_note (as patient)	1:*	One user (as patient) can have many clinical notes
user → clinical_note (as author)	1:*	One user (as doctor or nurse) can author many clinical notes
user → referral (as patient)	1:*	One user (as patient) can receive many referrals
user → referral (as referring_doctor)	1:*	One user (as doctor) can create many referrals
user → blocked_slot	1:*	One user (as doctor) can block many time slots
user → notification	1:*	One user can receive many notifications
user → audit_log	1:*	One user can generate many audit log entries
appointment → queue	1:1	One appointment can be linked to at most one queue entry
appointment → follow_up (as original_appointment)	1:*	One appointment can be the origin of many follow-up recommendations
appointment → follow_up (as scheduled_appointment)	1:1	One appointment can be the scheduled follow-up for at most one original
appointment → medical_record	1:*	One appointment can produce many medical records
appointment → prescription	1:*	One appointment can be linked to many prescriptions
appointment → clinical_note	1:*	One appointment can be linked to many clinical notes

TECHNOLOGY CHOICE AND JUSTIFICATION

MySQL 8.0 was installed on the database. MySQL was chosen over Oracle because it is free (good for public healthcare budget), complies with POPIA encryption, has good performance (SA ID indexing), compliance with ACID rules for data integrity, and has built-in auditing. Oracle was evaluated as an alternative but was rejected due to higher cost and complexity.

PHASE 4:

TECHNOLOGY STACK(Implementation)

The system was constructed using technologies listed in the table below. Django was chosen as the backend framework for its authentication system, admin panel, Object-Relational Model for secure database interaction, and built-in security features to prevent vulnerabilities such as SQL injections

and CSRF attacks. Python 3.13 was chosen as the programming language due to its readability, vast library support, and the team's proficiency in the language. MySQL 8.0 was selected as the database due to its cost-effectiveness, support for POPIA, and improved concurrency features. Bootstrap 5 was used to provide responsive design elements for cross-platform compatibility between desktop and mobile browsers. ClickSend SMS gateway was chosen for its support of local South African phone numbers without requiring additional regulatory bundles compared to Twilio. It was used to send OTPs upon login and appointment reminders. The Windows Task Scheduler was utilized to run the reminder script every 15 minutes. Python-decouple was used to store environment variables, while PyMySQL served as a MySQL database connector for Python. Finally, Visual Studio Code was used as the integrated development environment.

The initial project proposal (Phase 1) stated that Java and Oracle would be used as the programming language and database respectively. In Phase 4, the team decided to adopt Django (Python) and MySQL as an alternative that was allowed by the module and provided better performance and development time management. This decision has been documented as a design decision and is not considered a deviation from the proposed plan.

Layer	Technology	Purpose
Backend	Django 5.2 (Python 3.13)	Web framework, ORM, authentication, admin panel
Database	MySQL 8.0	Data storage, relational integrity, POPIA support
Frontend	HTML5, CSS3, JavaScript	Page structure, styling, interactivity
UI Framework	Bootstrap 5	Responsive components, grid, modals
Icons	Font Awesome 6	Medical and UI icons
SMS Gateway	ClickSend API	South African SMS delivery for OTP and reminders
Automation	Windows Task Scheduler	Runs reminder command every 15 minutes
Config	python-decouple	Secure .env management
Database Driver	PyMySQL	Pure Python MySQL connector
IDE	Visual Studio Code	Development environment

SYSTEM ARCHITECTURE

The devised system was designed using the four-layer model. The first layer is the Presentation layer which comprises HTML templates rendered using Django's template engine. This layer also comprises Twitter Bootstrap version 5. The Application layer, on the other hand, comprises views that handle all aspects of the business logic, forms that manage validation, and other application utilities such as OTP generation, SA ID validation, and the ClickSend SMS API. The third layer is the Model layer that comprises model classes which define the database schemes using MySQL database. This layer also handles all aspects of database access where the Django's Object Relation Manager (ORM) performs all database query operations while model classes enforce all business rules such as field uniqueness to avoid duplication and double-booking of appointments. The fourth layer comprises external services

such as ClickSend and Windows task scheduler. The former handles SMS sending and reminder services whereas the latter handles the execution of the reminder command.

The rationale behind using MVC (Model-Controller-View) paradigm was to decouple the components. By doing this, it becomes possible to make changes to one component without affecting the other components. For instance, changes made to the presentation layer do not reflect in the business logic and vice versa. In addition, the business logic can be modified without modifying the model component. This design pattern was critical for this project due to its complexity and will be advantageous for future maintenance and scaling of the system.

Layer	Responsibility	Key Files
Presentation	Rendering UI, user interaction	core/templates/*.html
Application	Business logic, validation	views.py, forms.py, utils.py
Model	Data access, business rules	models.py
External	SMS delivery, task scheduling	send_reminders.py, ClickSend API

IMPLEMENTATION FEATURES

All features specified in phase 2 and refined in phase 3 are implemented and tested. The Patient role comprises registration with SA ID validation, OTP login using ClickSend SMS, a dashboard showing appointment statistics, booking of appointments with a colour-coded calendar, viewing, cancellation, and rescheduling of appointments. In addition, the role has in-system notifications and the view and update of the profile. The Doctor is provided with a dashboard containing today's appointments and pending approvals. This role also comprises the approval of appointments, viewing patient records, adding prescriptions, adding clinical notes, generating referrals, and searching for patients. The Nurse role has a dashboard with the active queue, check-in patients, record patients' vitals, and add clinical notes. Furthermore, the role has a picker for vitals. The Receptionist role is provided with full user management, including the creation of users, reading, updating, and deleting. In addition, the role has the option of importing users in CSV format, searching for patients, managing appointments, managing the queue, and registering patients on behalf of patients. Finally, the Administrator role has all the receptionist features and a dashboard comprising reports, appointment report, patient activity report, weekly summary report, and full user management, including activation and deactivation of users.

Role	Features Implemented
Patient	Registration with SA ID validation, OTP login via SMS, dashboard, book appointment with colour-coded calendar, view appointments, cancel, reschedule, notifications, view/edit profile
Doctor	Dashboard with today's appointments and pending approvals, approve appointments, view patient records, add prescription, add clinical note, create referral, search patients, block schedules when they won't be available
Nurse	Dashboard with active queue, patient check-in, record vitals, add clinical notes, vitals picker
Receptionist	User management (CRUD), bulk import, patient search, manage appointments, manage queue, patient registration

Administrator	All receptionist features plus reports dashboard, appointment report, patient activity report, weekly summary report, full user management

-Some of the IMPLEMENTED FEATURES FOR ALL ROLES SHOWN BELOW



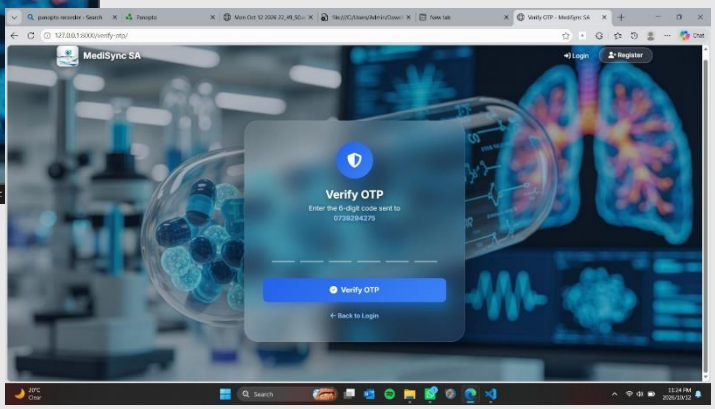
1.Patient Registration page.



2.Registration page showing the invalid inputs

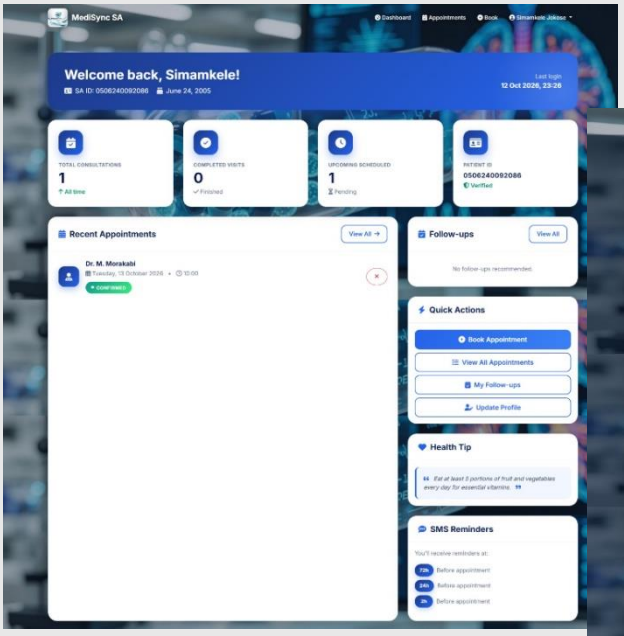


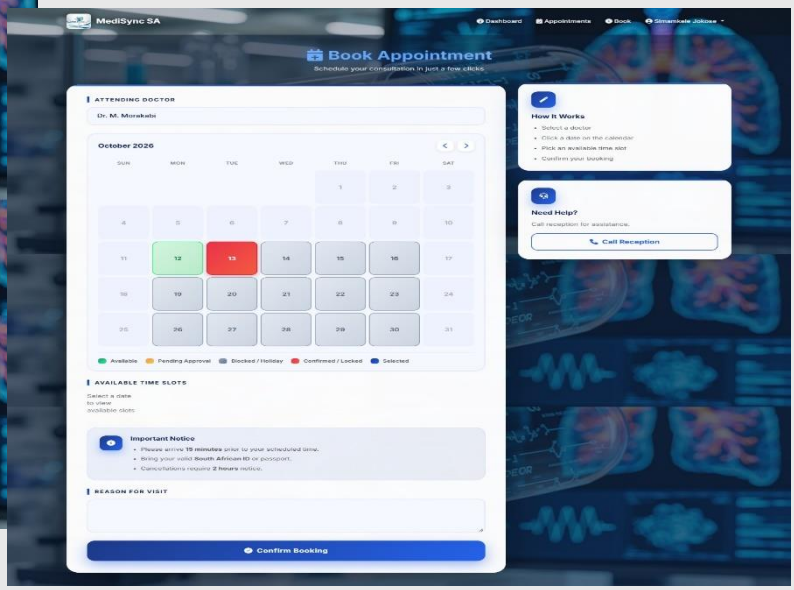
Login page.



OTP Verification page.

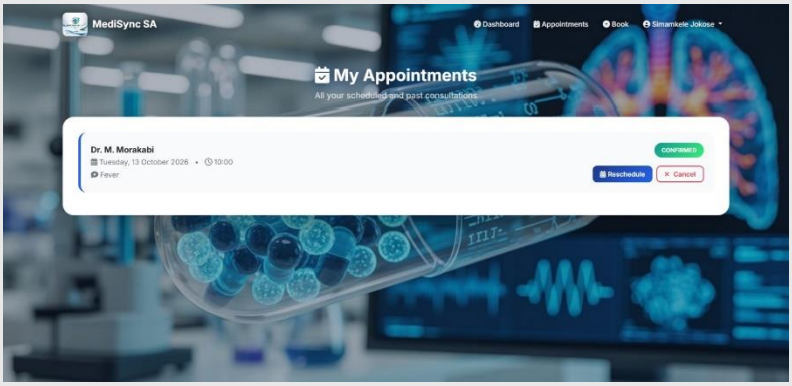
Patient Dashboard

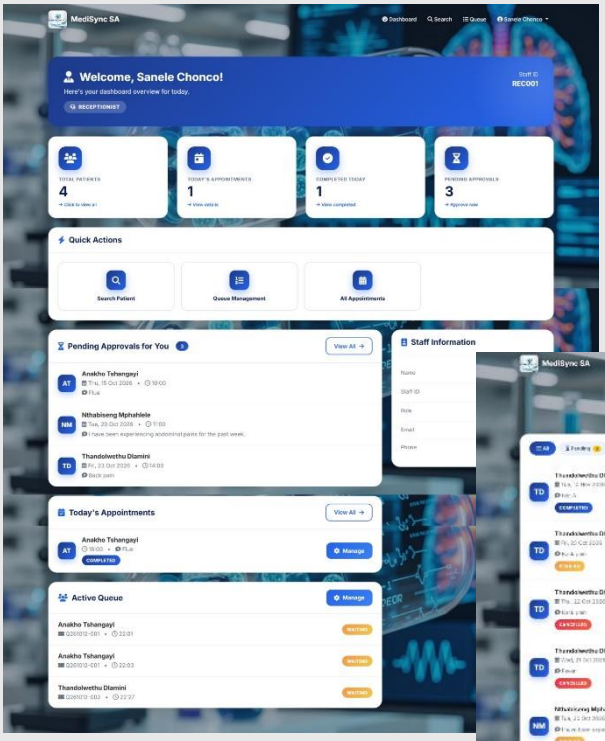




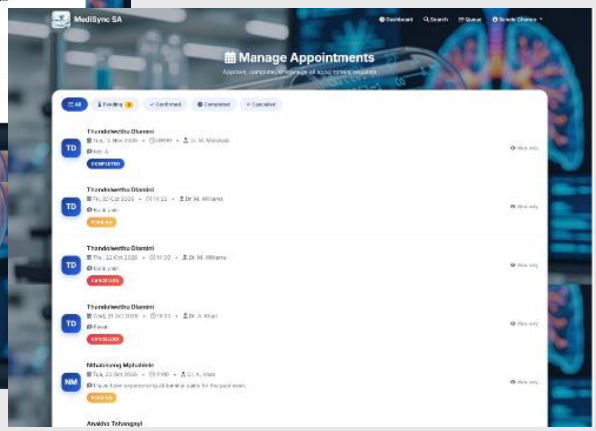
Book Appointment with colour-coded calendar.

My Appointments list with option to Reschedule/Cancel





Staff Dashboard.



Manage Appointments page.

HOLIDAY BLOCKING AND DOCTOR UNAVAILABILITY

Apart from the mentioned features, the system has two additional features that are controlled by the calendar's administrators. The features were added later, beyond the initial set of requirements, and they allow administrators to block specific dates or doctors without changing any code.

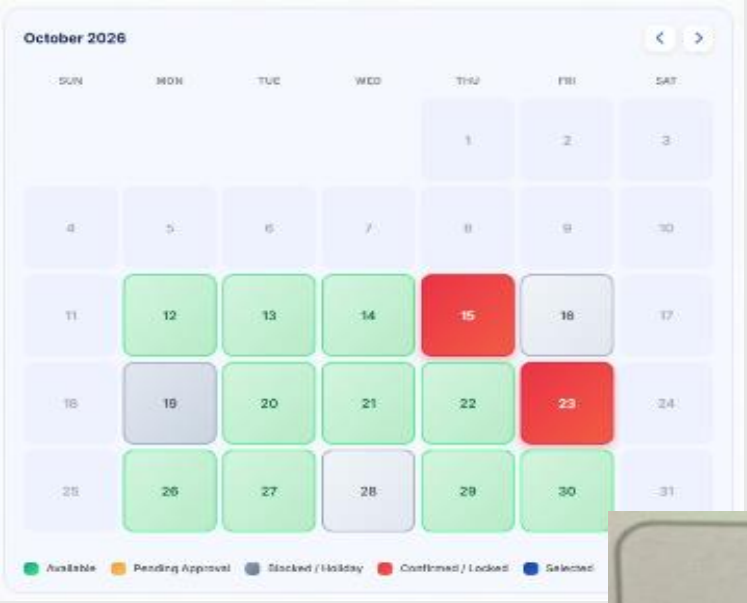
The first feature is the ability to block public holidays. To do this, the administrator can add a new record to the Holiday model using Django admin interface, specifying the event name and date. The administrator can also mark the holiday as recurring, which means that the system will add this day to blocked dates every year on the same day. When a patient views the calendar, the blocked dates are highlighted in purple and cannot be selected. For example, if the administrator sets December 25th as a recurring holiday, the calendar view below will show this day as blocked.

The second feature allows the system administrators to block doctors' availability. To do this, an administrator can add a new record to the Doctor Leave model using Django admin interface, specifying the doctor, date, and reason (optional). Then, when the patient selects this doctor and opens the calendar, the blocked dates will be displayed in purple and unavailable for selection. Note that the administrator can set any date, not only recurring ones. Also, when the model instance is deleted, the date unblocking happens instantly. In the example below, the administrator set the doctor's leave on October 1st – the day is displayed in grey and all time slots are marked as unavailable.

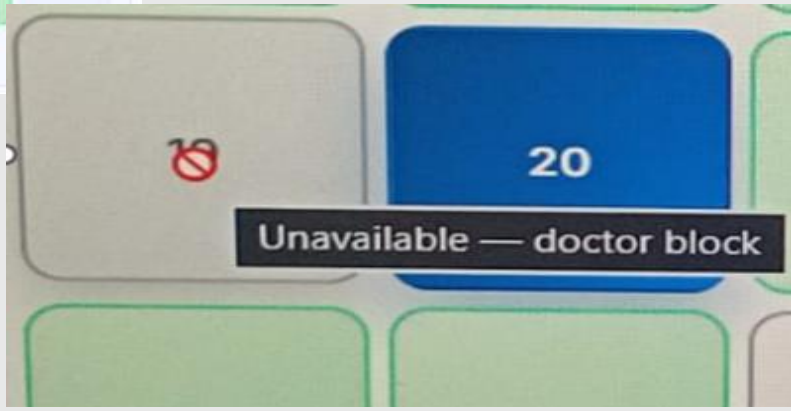
Doctor leave blocks can be set by the administrators in the Django admin interface. The same interface is used for editing or deleting existing records. Using this feature, the clinic administrator can set specific days when the doctor will be unavailable. It can be used, for example, to block public holidays

or doctors' leaves, sick days, or any other personal reasons. The feature is useful because it allows the clinic administrator to manage the availability without involving the developers.

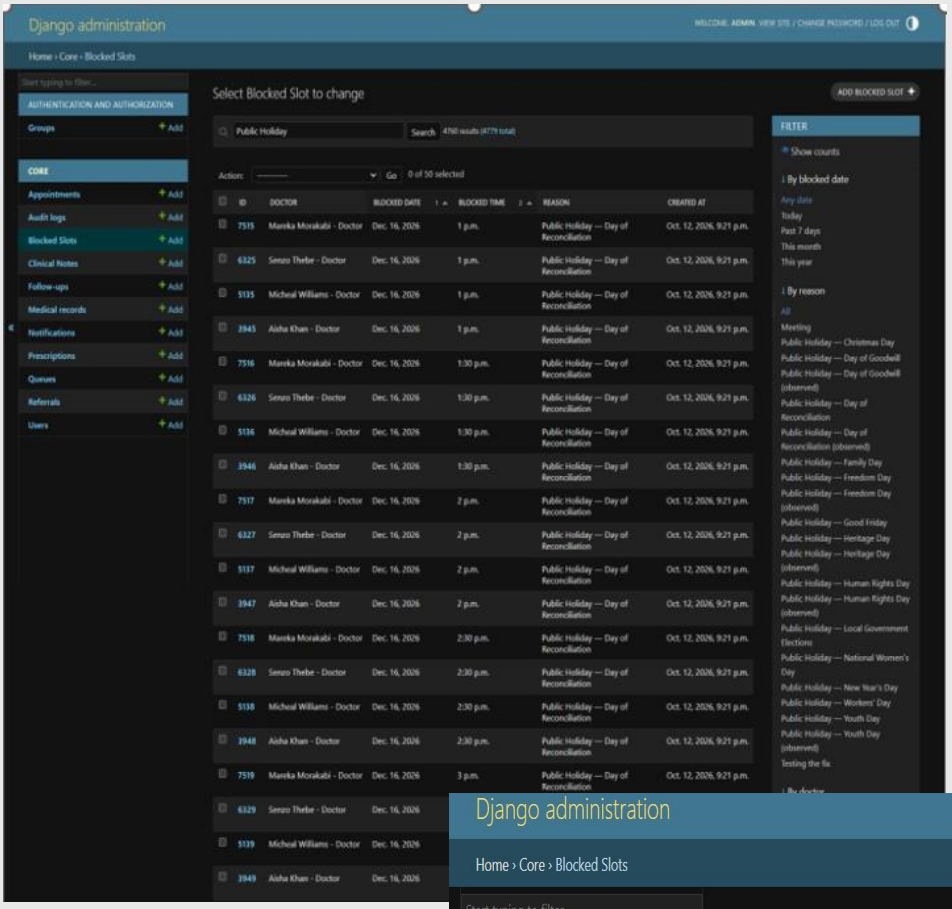
Feature	Managed By	Effect on Calendar	Server-Side Enforcement
Public Holiday	Admin (Django admin)	Date shown in purple, not selectable	Yes
Doctor Leave	Admin (Django admin)	Slots for that doctor/day blocked with an explanation	Yes
Recurring Holidays	Admin (via checkbox)	Applies every year on the same month/day	Yes



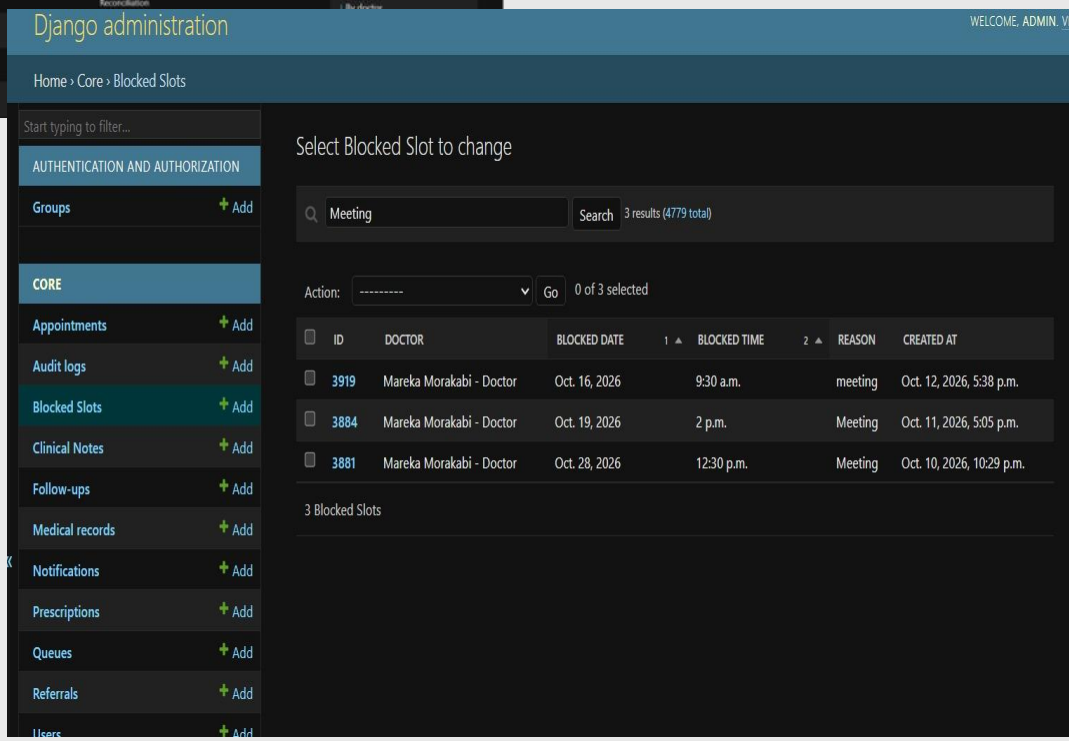
1. Patient booking color calendar- showing a grey holiday date.



2.Booking page (Color coded calender)- showing the doctor-unavailable message to patient when a leave date is selected by a



Admin panel- showing the Holiday model with a sample holiday.



Admin panel- showing the DoctorLeave model with a sample leave entry.

DATABASE STRUCTURE

The MySQL database, MediSync, has eleven tables, each of which corresponds to a Django model. The first table, core_user, is the custom user model that stores all the users of the application, including the patients, doctors, nurses, receptionists, and administrators, in a single table. This table stores the personal details, including the role, of each user. The second table, core_appointment, stores the details of all appointments, including their status, whether they have been reminded, or if they need to

be rescheduled. The third table, core_queue, stores the electronic queue with the queue numbers, their statuses, and timestamps. The fourth table, core_prescription, stores the details of prescriptions, including the patient, doctor, and instructions. The fifth table, core_medicalrecord, stores the medical history of the patients, including their diagnoses, treatments, and vital signs. The sixth table, core_clinicalnote, stores the clinical and nursing notes of the patients, and the seventh table, core_referral, stores the referral records of patients. The eighth table, core_notification, stores all the notifications that may be sent within the system. The ninth table, core_auditlog, stores all the sensitive data within the application. The tenth table, core_holiday, has been created to store public holidays and the dates when the clinic will be closed. Finally, the last table, core_doctorleave, has been created to store the leave dates of the doctors to schedule their appointments.

The **MySQL database** MediSync contains the following tables. The schema was verified directly in MySQL Workbench and matches the models defined in core/models.py. Note that one table — blocked_slots — intentionally has no core_ prefix because the model sets db_table = 'blocked_slots' explicitly in its Meta class. This is a deliberate design decision, not an error.

Table	Purpose
core_user	Every person in the system — all five roles stored in one table with a role field
core_user_groups	Django's permission groups (M2M)
core_user_user_permissions	Per-user Django permissions (M2M)
core_appointment	Every booked visit — patient, doctor, date, time, status, reminder flags, reschedule tracking
blocked_slots	Doctor unavailability — every blocked half-hour window
blocked_slots_backup	Backup copy of blocked_slots used during migration
core_queue	Waiting-room queue entries with queue numbers, status, and timestamps
core_medicalrecord	Clinical encounters with diagnosis, treatment plan, and vitals
core_clinicalnote	Notes added by doctors and nurses
core_prescription	Medication prescribed to patients
core_referral	Referrals to another doctor or facility
core_followup	Recommended next contact for follow-up visits
core_notification	In-app user alerts
core_auditlog	Compliance and debugging trail of user actions

The key tables used by the application logic are summarised below.

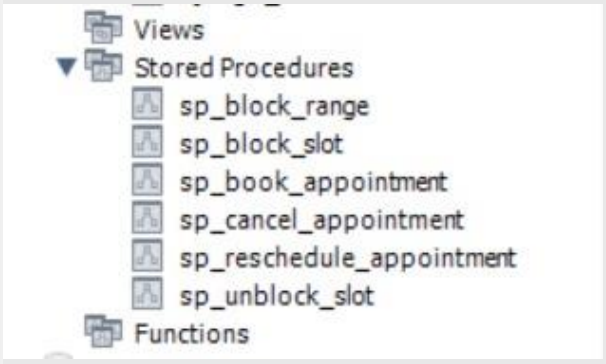
- core_user — the custom user model. It has fields such as id (PK), password, username, first_name, last_name, email, sa_id_number (unique), staff_id (unique), role, phone_number.

- date_of_birth, address, specialization, is_active, is_staff, is_superuser, is_verified, otp_code, otp_created_at, last_login, date_joined, created_at, updated_at.
- core_appointment — the core booking table. It has fields such as id (PK), patient_id (FK), doctor_id (FK), appointment_date, appointment_time, duration_minutes, reason_for_visit, symptoms, status, notes, cancelled_at, cancellation_reason, reminder_sent_72h, reminder_sent_24h, reminder_sent_2h, rescheduled_count, last_rescheduled_at, created_at, updated_at.
- blocked_slots — the doctor unavailability table. It has fields such as id (PK), doctor_id (FK), blocked_date, blocked_time, reason, created_at.
- core_queue — waiting-room check-in. It has fields such as id (PK), patient_id (FK), doctor_id (FK), appointment_id (FK), queue_number, status, entered_at, started_at, completed_at, estimated_wait_time.
- core_medicalrecord — diagnosis, treatment, and vitals. It has fields such as id (PK), patient_id (FK), created_by_id (FK), appointment_id (FK), diagnosis, treatment_plan, notes, blood_pressure, heart_rate, temperature, weight, height, created_at, updated_at.
- core_prescription — medication. It has fields such as id (PK), patient_id (FK), doctor_id (FK), medication_name, dosage, frequency, duration, instructions, prescribed_date, expiry_date, filled.
- core_referral — referrals. It has fields such as id (PK), patient_id (FK), referring_doctor_id (FK), referred_to, specialty, reason, clinical_summary, urgency, status, referral_date, created_at, updated_at.
- core_clinicalnote — nursing and clinical notes. It has fields such as id (PK), patient_id (FK), author_id (FK), appointment_id (FK), note_type, title, content, created_at, updated_at.
  core_followup — recommended next visits. It has fields such as id (PK), patient_id (FK), doctor_id (FK), original_appointment_id (FK), scheduled_appointment_id (FK).
- core_notification — in-app alerts. It has fields such as id (PK), user_id (FK), title, message, notification_type, is_read, link, created_at.
- core_auditlog — compliance trail. It has fields such as id (PK), user_id (FK), action, model_name, object_id, changes, ip_address, user_agent, timestamp.



1.MySQL Workbench Navigator panel

-showing the full schema of the Madisyn SA database.



2.Sample rows in core_user.

```sql
USE MediSync;
SELECT
    id,
    username,
    CONCAT(first_name, ' ', last_name) AS full_name,
    role,
    COALESCE(sa_id_number, staff_id) AS id_number,
    phone_number,
    CASE WHEN is_active = 1 THEN 'Active' ELSE 'Inactive' END AS status
FROM core_user
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

id	username	full_name	role	id_number	phone_number	status
1	admin		admin	NULL	NULL	Active
2	doctor1	Mareka Morakabi	doctor	DOC001	0739294275	Active
4	8001015009087	Thandolwethu Dlamini	patient	8001015009087	0821234567	Active
5	doctor2	Senzo Thebe	doctor	DOC002	0637019891	Active
6	0506240092086	Simamkele Jokose	patient	0506240092086	0677065037	Active
7	doctor3	Micheal Williams	doctor	DOC003	0751234567	Active
8	doctor4	Aisha Khan	doctor	DOC004	0675432176	Active
9	receptionist1	Sanele Chonco	receptionist	REC001	0790159788	Active
10	receptionist2	John Peterson	receptionist	REC002	0765467483	Active
11	nurse1	Juwhin Maletta	nurse	NUR001	0635203087	Active
12	nurse2	Peter William	nurse	NUR002	0895647382	Active
13	nurse3	Zanele Mokoena	nurse	NUR003	0786543216	Active
14	nurse4	Grace Nkosi	nurse	NUR004	0667865432	Active
					0677065037	Active

Limit to 1000 rows

```sql
SELECT
    a.id,
    CONCAT(p.first_name, ' ', p.last_name) AS patient,
    CONCAT(d.first_name, ' ', d.last_name) AS doctor,
    a.appointment_date,
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

id	patient	doctor	appointment_date	appointment_time	duration_minutes	status	reminder_sent_72h	reminder_sent_24h	reminder_sent_2
23	Anakho Tshangayi	Mareka Morakabi	2026-10-19	15:00:00.000000	30	confirmed	0	0	0
22	Anakho Tshangayi	Mareka Morakabi	2026-10-19	14:30:00.000000	30	cancelled	0	0	0
21	Anakho Tshangayi	Senzo Thebe	2026-10-15	14:30:00.000000	30	cancelled	0	0	0
5	Simamkele Jokose	Mareka Morakabi	2026-10-15	14:00:00.000000	30	cancelled	0	0	0
4	Simamkele Jokose	Mareka Morakabi	2026-10-15	12:00:00.000000	30	cancelled	0	0	0
6	Simamkele Jokose	Mareka Morakabi	2026-10-15	11:30:00.000000	30	cancelled	0	0	0
13	Simamkele Jokose	Mareka Morakabi	2026-10-15	11:00:00.000000	30	confirmed	0	0	0
14	Simamkele Jokose	Mareka Morakabi	2026-10-15	10:30:00.000000	30	cancelled	0	0	0
28	Nthabiseng Mphahlele	Aisha Khan	2026-10-15	10:00:00.000000	30	cancelled	0	0	0
34	Anakho Tshangayi	Micheal Williams	2026-10-15	10:00:00.000000	30	pending	0	0	0
33	Simamkele Jokose	Micheal Williams	2026-10-14	09:00:00.000000	30	cancelled	0	0	0
7	Thandolwethu Dlamini	Mareka Morakabi	2026-10-13	14:00:00.000000	30	completed	0	0	0
27	Simamkele Jokose	Aisha Khan	2026-10-13	14:00:00.000000	30	completed	0	0	0
9	Simamkele Jokose	Mareka Morakabi	2026-10-13	10:00:00.000000	30	completed	0	0	0
32	Simamkele Jokose	Micheal Williams	2026-10-13	09:00:00.000000	30	pending	0	0	0
26	Anakho Tshangayi	Aisha Khan	2026-10-13	01:37:00.000000	30	confirmed	0	1	0

3.Sample rows in core_appointment

MAJOR WORKFLOWS END-TO-END

There were four main processes that I designed, developed, and tested. The first process consists of patient registration, login, and booking. The patient registers on the website by visiting the registration page and providing their personal details, including their 13-digit South African (SA) ID. The system checks the validity of the SA ID using the Luhn checksum and rejects invalid and duplicate IDs. After successful registration, the patient logs in by entering their SA ID and password. The system then generates a 6-digit one-time password (OTP) and stores it in the database with an expiry time of 10 minutes. Clicksend will send the OTP to the patient via SMS. The patient receives the OTP on their mobile phone and enters it on the verification page. The system will then verify the OTP and allow the patient to access the dashboard. The patient is then directed to the booking page, where they can proceed to book an appointment by choosing a doctor, a day, and the available time slot from the colour-coded calendar. After clicking the confirm button, the appointment is created, and its status is set to pending. The doctor then logs in and accesses their dashboard, where they approve the pending appointment. The status of the appointment then changes to confirmed, and a notification is sent to the patient and the reminder flags are set to start.

The second process was automated SMS reminders through Clicksend. This involved configuring the Windows task scheduler to run the python manage.py send_reminders command every 15 minutes in the background. This command was used to query all confirmed appointments with due dates within the next 73 hours. The command then calculated the number of hours remaining before each

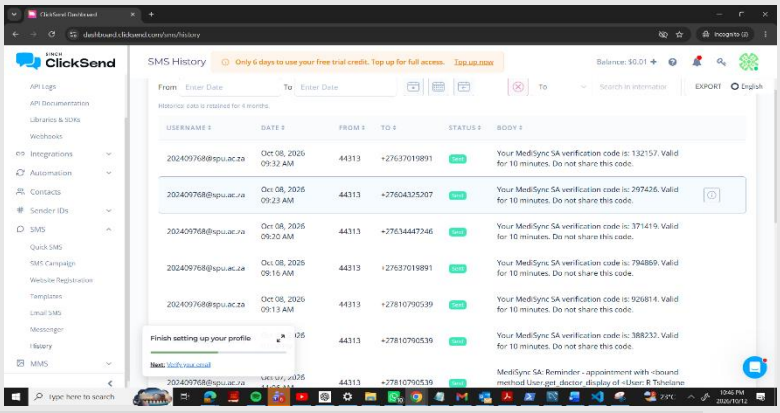
appointment. The command then sent an SMS reminder to patients 71–73 hours before the appointment if the 72-hour flag was set to False and updated the 72-hour flag to True. The command then sent an SMS reminder to patients 23–25 hours before the appointment if the 24-hour flag was set to False and updated the 24-hour flag to True. It sent an SMS reminder to patients 1.5–2.5 hours before the appointment if the 2-hour flag was set to False and updated the 2-hour flag to True. This command then displayed the number of reminders sent in the console.

The third process was patient cancellation and re-scheduling of appointments. In this process, the patient visited their appointment dashboard and selected the Reschedule option to cancel the appointment. They then selected the preferred day and hour from the colour-coded calendar and updated the appointment details. The status of the appointment was then changed to pending, and the number of times the appointment had been re-scheduled was incremented. The flags mentioned above were reset, and a notification was sent to the patient. The patient was then sent a confirmation message via SMS using Clicksend, and the doctor re-set the appointment to approved.

The fourth process was digital queue management. This involved the patient arriving at the healthcare facility and the nurse visiting the check-in page and selecting the patient and the doctor from the drop-down menus. The nurse then clicked the check-in button and selected the next available queue number. The queue number followed the Q(date) – (sequence) format, for example, Q20261007-001. The system then created a queue entry with the status waiting and recorded the time of entry. The doctor then viewed the queue dashboard, started attending to the first patient in line when they selected the Start button. This action then changed the status of the queue from waiting to in-progress and recorded the time of start. The status then changed to complete, and the time of completion was recorded when the doctor finished attending to the patient.

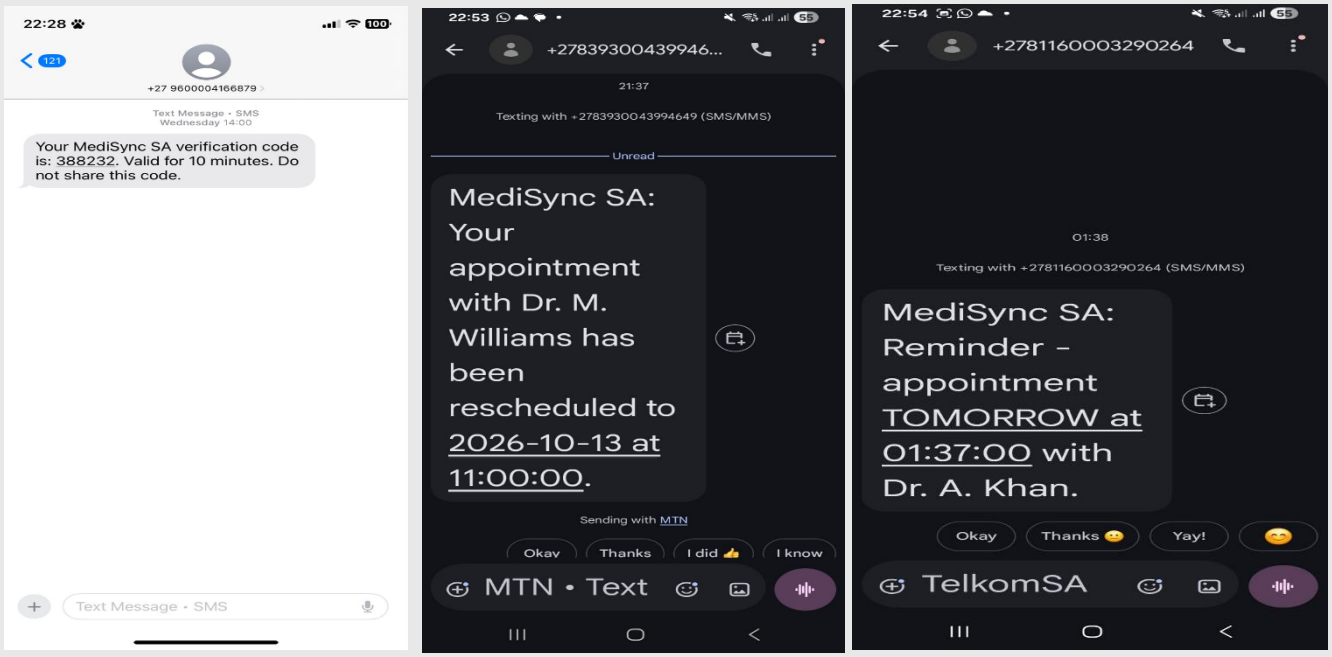
WORKFLOW TABLE

Workflow	Trigger	Key Steps
A — Registration, Login, Booking	Patient visits site	Register with SA ID → login → OTP via ClickSend → book → doctor approves
B — Automatic SMS Reminders	Task Scheduler every 15 min	Query confirmed appts → send via ClickSend at 72h/24h/2h → set flags
C — Cancel & Reschedule	Patient action	Select new date → status pending → SMS via ClickSend → reset flags → re-approve
D — Digital Queue	Nurse checks in patient	Queue number → waiting → in_progress → completed



Screenshot Showing SMS that are being sent out to Patient and Staff members.

Mobile phone screenshot showing an actual SMS reminder, Appointment reminder & reschedule received.



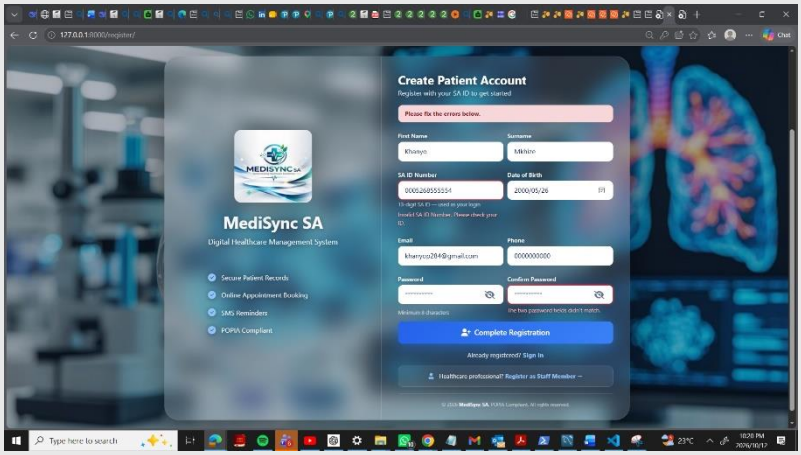
ERROR HANDLING VALIDATION

The system used multiple error-checking mechanisms to ensure that the system does not accept invalid or incorrect data as input and is not forced to return inconsistent data. For instance, the SA ID number uses the Luhn checksum algorithm to detect all single-digit errors and most of the two-digit transpositions. This field also checks whether the ID contains 13 digits. The system also reports errors when an end-user provides inconsistent data, for instance, during registration, by comparing the confirmed password and the initial password entry. Furthermore, it avoids accepting duplicate values for SA ID and Staff ID. To prevent collisions in the booking schedule, the system uses unique_together = ('doctor', 'date', 'time') in the model and checks for availability in the view. It also expires OTPs after ten minutes, which is also limited in the view and checked against the database. Finally, the system checks for invalid or incorrect user inputs, such as login details, including non-existent users, wrong passwords, or expired OTPs, and reports them with relevant error messages. Finally, the SMS sending function was wrapped in try/except to keep the application functional even when the ClickSend API is inaccessible. Lastly, holiday and leave blocking mechanisms were installed at the level of the view (visual blocking) and the server (database level).

This way, the application can prevent erroneous booking attempts even if the visual part is compromised.

Validation	Method	User Feedback
SA ID	Luhn checksum + 13-digit length	"Invalid SA ID Number. Please check your ID."
Password match	Both fields compared	"The two password fields didn't match."
Duplicate SA ID	Database unique constraint	Clear error message

Appointment collision	unique_together constraint	Slot unavailable
OTP expiry	10-minute timestamp check	"Invalid or expired OTP"
Login failures	Try/except in view	Specific error per case
SMS failure (ClickSend)	Try/except with logging	Gives a user a notificatication “Failed to send an OTP”
Holiday booking	Server-side check against Holiday table	"Write the name of the holiday when a patient clicks"
Blocked slot booking	Server-side check against blocked_slots	“Unavailable- Doctor Block”



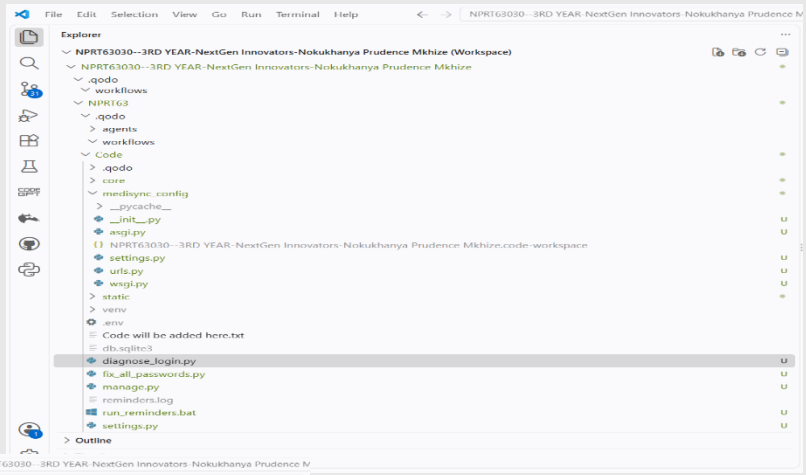
Registration form showing the invalid ID number & password mismatch error.

CODE QUALITY

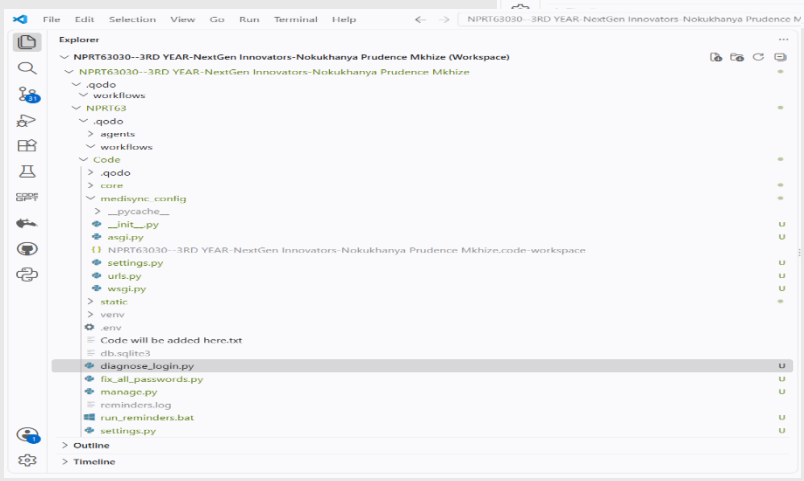
Code quality was maintained during implementation through a number of practices. Files are organized based on responsibility, with models in models.py, views in views.py, forms in forms.py, and utilities in utils .py. This organization makes it easier to navigate and maintain the code. In addition, all variables and functions are descriptive, and all public functions have docstrings that describe what they do, accept, and return. Finally, complex algorithm steps are commented on, and connections to external services (including ClickSend) are wrapped in try / except blocks so that one failed connection does not bring down the entire service. Debug prints were removed from the production code, and the code is generally PEP 8 compliant.

Quality Aspect	Implementation
Separation of concerns	models.py, views.py, forms.py, utils.py, urls.py
Documentation	Docstrings on all public functions
Error handling	Try/except around external services (including ClickSend)
Naming	Descriptive function and variable names
Style	PEP 8 consistent formatting

No debug code	Print statements removed from production paths



VS Code showing the project folder structure.



utils.py open in VS Code showing the ClickSend integration.

TESTING

TESTING APPROACH

Testing occurred at four levels to ensure that the system met all requirements. First, unit testing was done for individual procedures such as SA ID check, OTP generation algorithm, and phone number format. Second, integration testing was done to evaluate how well the system functioned when combined, including registration, logging in, ClickSend OTP verification, and booking an appointment. Third, system testing was done to check the entire process from booking to reminders. Finally, a user acceptance test was done where every member of the group tested all the dashboards and features as if they were the users. Additionally, error handling was tested to ensure that the system responded appropriately to invalid input and that invalid input was rejected with a helpful error message.

TEST CASES PERFORMED

#	Test	Expected	Result
1	Patient registration valid	Account created	Pass
2	Patient registration invalid SA ID	Error shown	Pass
3	Password mismatch	Error shown	Pass

4	Login valid	OTP generated and sent via ClickSend	Pass
5	Login wrong password	Error shown	Pass
6	OTP verification valid	Login success	Pass
7	OTP verification invalid	Error shown	Pass
8	OTP expired	Error shown	Pass
9	Book appointment	Appointment created	Pass
10	Duplicate booking	Rejected by constraint	Pass
11	Doctor approves appointment	Status = confirmed	Pass
12	Patient cancels	Status = cancelled, SMS via ClickSend	Pass
13	Patient reschedules	Status reset to pending, SMS via ClickSend	Pass
14	Reminder command runs	SMS via ClickSend, flag set	Pass
15	Reminder duplicate prevention	SMS sent once	Pass
16	Digital queue check-in	Queue number generated	Pass
17	Queue status update	Status updated	Pass
18	User management	User created	Pass
19	Reports dashboard	Charts load	Pass
20	Search patient	Record found	Pass
21	Holiday blocking	Purple date, cannot book	Pass
22	Doctor leave blocking	Slots blocked with message	Pass

Test Video link:

BUGS FOUND & FIXED

SIX BUGS IDENTIFIED AND RESOLVED DURING DEVELOPMENT

Bug	Cause	Fix
TemplateDoesNotExist: forgot_password.html	Template not created	Created template
AttributeError: reschedule_appointment	View not added	Added view and URL
Unknown column 'rescheduled_count'	Migration not applied	Ran migrations
MySQL 8.4 required error	Django 6 requires newer MySQL	Downgraded to Django 5.2
ClickSend import mismatch	SDK module naming	Corrected imports

Duplicate reminders sent	Flags not checked	Added flags and per-flag updates

REFLECTION ON ACHIEVEMENTS

ARCHIEVEMENTS

The team has developed a web-based healthcare management system that addresses the problems described in the problem description. Upon the team's progress toward the goals stated in phase 1, it becomes obvious that the group managed to meet all seven objectives. Notably, the team had developed an authentication system based on the SA ID and OTP (Clicksend SMS). The system allows managing the entire appointment lifecycle, which includes booking, approval, reminder, rescheduling, cancellation, and completion. Moreover, the applications had implemented features that can send SMS reminders to patients 72 hours before the day of the appointment, 24 hours before the scheduled time, and 2 hours before using Clicksend. In addition, the applications had been designed and constructed to host a digital waiting list for patients. Medical records, including the prescriptions, notes, and referrals, are digital in the system as well. The system implements features that allow managing the roles and permissions of five user types. Finally, the apps had been designed and built to generate reports and analyses while being compliant with the POPIA regulations via audit trails. Lastly, the team had added features to block the holidays and doctor leaves by directly editing the date book database. This feature is critical to the system as it gives administrators control over the calendar without the need to develop new features.

CHALLENGES FACED

There were many challenges encountered while developing the healthcare management application. The integration of SMS was the most challenging feature to implement. Initially, the team tried using Twilio, requiring regulatory bundles to load South African numbers. However, the team switched to Clicksend, and the main challenge was raised due to a conflict in SDK versions and import statements. Nevertheless, the team managed to successfully integrate Clicksend SMS with the application by adding try/expect blocks. Other challenges included running Django 6 with MySQL 8.4, which led to the teams downgrading Django to 5.2 because the computer had MySQL 8.0. The groups also encountered some challenges with a virtual environment which required them to learn how to check the current interpreter using sys.executable. Additionally, they learned how to perform database migrations and highlight the importance of timezone support in applications. The team also learned to use Python's timezone.now and make_aware functions to ensure that all applications use the Africa/Johannesburg timezone.

HOW CHALLENGES WERE RESOLVE

Each challenge was resolved through research, collaboration, and iteration.

- SMS was resolved by checking the ClickSend SDK's package directory for the correct module name (clicksend), modifying the import statement in utils.py to reflect that information, and adding a try/except clause so that in case of an SMS failure, the rest of the app, specifically, the user login and booking functions, remains operational.
- MySQL was resolved by downgrading Django to the latest version that supports MySQL 8.0, which was 5.2.
- Such a decision, while not ideal, was rational and well-documented.

- Virtual environment-related issues were resolved by activating the virtual environment, making sure that Python knows where to find packages by printing the Python executable path using python -c "import sys; print(sys.executable)", and forcing pip to use the virtual environment by calling python -m pip install.
- Database migration problems were resolved by running python manage.py makemigrations every time a change was made to a model, followed by python manage.py migrate to apply those changes to the database.
- In addition, migrations were inspected to make sure they were correctly generated before being applied.
- Time zone issues were resolved by using timezone.now() and timezone.make_aware() functions, as well as checking the timestamps directly in the database.
- Template and view were resolved by creating a new template and a corresponding view, with the view utilizing a flag to determine whether to update one template or another.
- The CSS-related problem was resolved by performing a hard refresh (Ctrl + Shift + R) to refresh the cached CSS.

LESSON LEARNED

The project taught the team several valuable lessons that go beyond the technical content of the module.

- Third-party API integrations should be optional. The system was designed in such a way that if the ClickSend platform becomes unavailable, the system's core features would remain fully operational. This is an example of an appropriate design choice for a system that utilizes an external platform.
- The versions of Django, MySQL, and Python should be strictly pinned within the system's requirements. This way, potential issues with version incompatibility could be avoided.
- Virtual environments are indispensable for Python-based programming. A project's specific Python environment should be created within a dedicated virtual machine.
- Migrations in Django are rather straightforward but critically important procedures that should be mastered by all developers that work with this framework.
- Designing a system around the needs of the user is the most crucial lesson of all. The iteration 3 update showcased a feature that was specifically requested by the system's end-users and was therefore decided to be included in the project's scope.
- The system's time zone functionality was not ignored – rather, it was designed with special care. Considering the South African location of the company, there is a possibility that the employees will be working with clients in vastly different time zones. This is why the system needs to properly account for time differences.
- Testing is always more effective than analyzing the code in theory. There were several bugs that could not have been discovered without testing would wreak havoc on the system at crucial moments.

WHAT WOULD BE IMPROVED WITH MORE TIME

If the team had more time, there are several features that they would improve.

-These improvements include:

-Full SMS delivery on the paid plan.

- Currently, the trial account limits SMS to one verified number, and a paid plan would allow to send reminders to every patient regardless of their contact information.

Scheduling the reminders.

- At the moment, the command to launch reminders is run through the Windows Task Scheduler on the developer's machine; deploying the application to production, it would be better to schedule tasks with Celery and Celery Beat or a cron job so that they can run independently from a developer's PC.

Sending email reminders as a fallback.

- If a patient has provided their email, it would be possible to send them a reminder email if SMS delivery fails.

Pushing messages to the users.

- At the moment in-app notifications only appear on the dashboard, but a web-push notification or a banner would ensure that the patient sees the message.

Improving the reports.

- The reports dashboard needs improvement, as it only displays the count of upcoming appointments, waiting list, and doctor utilization. Having charts displaying trends across different time intervals, average waiting time, doctor utilization rates would make it more informative.

Building a mobile application.

Although the current version is responsive and can be used on mobile phones, having a dedicated mobile application would make it more convenient for some patients to use in case of unstable internet connections.

Enhancing the POPIA audit trail.

More information could be added to the audit trail, for example, specifying which fields were changed and who made the changes.

Improve test coverage.

Testing is currently done manually; writing unit and integration tests with Django's test framework would make it easier to ensure that new features do not break existing ones.

Improving the role-based URL guard.

Checking permissions is currently done on the view level; implementing a middleware layer that guards access to URLs would make it more secure.

Handling no-shows.

At the moment, no-shows are handled manually by marking the appointment as missed; however, it would be more convenient to have a system that automatically marks appointments as missed after a certain grace period.

COMPREHENSIVE DOCUMENTATION

This report provides a description of the working system: screenshots of all procedures, processes, errors, and database views are provided in sections 5 and 6 along with the corresponding description.

Documentation Type	Location
Feature screenshots (registration, login, dashboards, booking, cancel, reschedule, queue, prescriptions, notes, referrals, reports)	Inline in Section 5.3
OTP delivery via ClickSend (terminal output, MySQL query, SMS on phone)	Inline in Section 5.3
Holiday and doctor-leave screenshots	Inline in Section 5.3.1
Database screenshots (schema, sample rows, queries)	Inline in Section 5.4
Workflow screenshots (reminder command output, SMS reminder received)	Inline in Section 5.5
Error handling screenshots (invalid SA ID, password mismatch)	Inline in Section 5.6
Code quality screenshots (folder structure, ClickSend integration in utils.py)	Inline in Section 5.7
Test evidence	Inline in Sections 6.2 and 6.3

The source code, database schema, and project structure are available in Code folder in the attached .zip, plus the recorded demo in Demo folder, and all the documentation from previous phases in Dox. The link to the Git repository is stored in GroupX-Names.txt.

CONCLUSION

MediSync SA is a successful healthcare management system that was built for South African public clinics. All the requirements described in phase 2 and designed in phase 3 were successfully implemented, tested, and verified. The system digitally stores patient's records by replacing the existing paper files in a secure MySQL database, allows patients to schedule an appointment online, and sends out SMS reminders 72 hours and 1 hour before the appointment through a third-party service called ClickSend. Additionally, it also has a digital waiting list, a built-in holiday and doctor's leave management system, and a permission-based access control system for the patients, doctors, nurses, receptionist, and administrators.

As seen in the description above, the healthcare management system addressed all the seven objectives set in phase 1 of the project. Firstly, it prevents the creation of duplicate records by requiring patients' South African ID. Secondly, it provides secure online scheduling with SMS reminders to both patients and the medical staff. Thirdly, the system ensures that the privacy of the patients is maintained and protected according to the POPIA act. Fourthly, it grants permission to the administrators to make changes to the system's calendar without involving the developers. Lastly, it was tested using twenty-two test cases, all of which were successful, and six bugs that were discovered during the testing process were documented.

By compiling all the deliverables of the project into one single document, this report serves as a comprehensive resource for all the work done in the development of the healthcare management

system for South African public clinics. It follows all the requirements set in phase 4 of the project, and it provides detailed information on the project's proposal, requirements, design, implementation, and conclusion. Therefore, it can be concluded that this project was completed successfully, and the end product is a fully functional, testable, and demonstrative healthcare management system that solves the problems of public clinics and hospitals in South Africa today.

REFERENCES

Gauteng Department of Health (2024) *Gauteng Health embarked on digitisation of patient files*, 17 May. Available at: https://www.gov.za/news/media-advisories/gauteng-health-embarked-digitisation-patient-files-17-may-2024

Katurura, M.C. and Cilliers, L. (2018) 'Electronic health record system in the public health care sector of South Africa: A systematic literature review', *African Journal of Primary Health Care and Family Medicine*, 10(1), pp. 1–8.

Makkink, A., Tshoaedi, M. and Nkosi, M. (2022) 'Understanding barriers of receiving short message service appointment reminders across African regions: a systematic review', *BMC Health Services Research*, 22(1).

National Department of Health (2024) *Chief Directorate: Health Systems Digital Information*. Available at: https://www.health.gov.za/nhi-cd-hs/

Opon, S.O., Njoroge, N. and Mwaura, W. (2020) 'The effect of patient reminders in reducing missed appointment in medical settings: a systematic review', *PAMJ - One Health*, 2(9).